

PAYTM PRE-FLIGHT

Updated Product Requirements + Technical Requirements

Single Source of Truth — Hackathon MVP v2

Paytm Build for India AI Hackathon — Bengaluru Edition

Primary journey: Personal-loan application preparation

Build constraint: ~8 hours

Status: Authoritative implementation contract

Paytm Pre-Flight

Product Requirements Document (PRD) + Technical Requirements Document (TRD)

Single Source of Truth — Hackathon MVP v2

Project: Paytm Pre-Flight

Team: Akagami / Rogue

Hackathon: Paytm Build for India AI Hackathon — Bengaluru Edition

Build constraint: ~8 hours

Primary journey: Personal-loan application preparation

Document status: Updated single source of truth for product, engineering, UX, demo, and AI-agent implementation

0. Executive Product Definition

0.1 One-line definition

Pre-Flight is an intelligent pre-submission financial application preparation, verification, and translation layer.

It helps a customer understand what they have provided, what is missing, what may need clarification, where important information came from, and what they can do next — before submitting a financial application.

0.2 Core product promise

Transform:

“I want to apply for a loan. What do I need?”

into:

“Here is what you have provided, here is what is still needed, here is what needs clarification, here is the evidence behind it, and here is your next action.”

0.3 Core loop

UNDERSTAND → EXTRACT → NORMALIZE → VERIFY → EXPLAIN → ACT → RE-VERIFY

The product should feel like an **application linter / pre-flight check** rather than an AI chatbot.

0.4 Three principles that define the product

UX principle

VALUE BEFORE DATA

Give the customer useful preparation information before asking for sensitive documents.

Technical principle

AI FOR INTERPRETATION, DETERMINISTIC LOGIC FOR VERIFICATION

AI interprets natural language and documents. Application logic performs critical comparisons, completeness checks, mappings, and status decisions.

Architectural principle

CANONICAL USER PROFILE + INSTITUTION-SPECIFIC CONFIGURATION

Capture the user's information once in a normalized representation, then map it to institution-specific requirements.

Product principle

PREPARE BEFORE YOU SUBMIT

Pre-Flight is a preparation layer, not a lender.

PART 1 — PRODUCT REQUIREMENTS DOCUMENT

1. Product Overview

1.1 Problem Statement

Applying for a financial product can be unnecessarily difficult.

A customer may know:

“I need ₹3 lakh for 24 months.”

But the actual application journey may require the customer to understand:

- what information is required
- which documents may be required
- why a particular document is being requested
- what information has already been provided
- what information is missing
- whether declarations and supporting evidence are consistent
- where an extracted value came from
- what needs attention
- what should happen next

This creates several recurring frictions:

1. Information overload

Long forms and document lists can make it difficult to understand what matters to the current application.

2. Repeated data entry

Relevant information may already exist in documents, yet the customer may have to enter it again.

3. Missing information

A customer may begin an application without realizing that an important field or supporting document is missing.

4. Information inconsistency

A customer declaration can differ from information in a supporting document.

Example:

- Declared income: ₹60,000/month
- Salary-slip net salary: ₹48,000/month

Pre-Flight should surface:

⚠ NEEDS CLARIFICATION

It should not decide whether the customer is eligible.

5. Poor explanation

Customers may see that something needs attention without understanding:

- what happened
- where the information came from
- why it may matter
- what they can do next

1.2 Opportunity

Pre-Flight creates value before submission by converting an unstructured customer request into a transparent preparation state.

The customer can see:

- information already captured
- information still missing
- documents they may need to prepare
- information that has been verified
- information that remains unverified
- inconsistencies requiring review
- evidence/provenance for important values
- the next procedural action

The goal is not to make a lending decision.

The goal is to make the customer better prepared before entering the official application process.

2. Product Vision

Pre-Flight sits between **customer intent** and **official financial application submission**.

It is an intelligent preparation, verification, and translation layer.

2.1 What Pre-Flight is

- an application preparation assistant
- an application linter
- a structured information normalizer
- a document-aware verification layer
- an explainability/provenance layer
- a next-action guide

2.2 What Pre-Flight is not

Pre-Flight is not:

- an AI loan approval system
- a loan prediction system
- a credit scoring system
- an AI financial advisor
- a lender
- a lender marketplace
- a chatbot that recommends which loan to take
- an autonomous application submission agent
- an AI that simply extracts document text

2.3 Boundary between AI and application logic

AI is responsible for

- natural-language interpretation
- extracting structured facts from user messages
- extracting relevant facts from documents
- generating plain-language explanations
- translating application terminology into understandable language

Deterministic application logic is responsible for

- schema validation
- canonical-field normalization
- institution-field mapping
- required-field checks
- required-document checks
- comparisons
- discrepancy calculations
- status transitions
- reconciliation rules
- deciding whether a check is `VERIFIED`, `PROVIDED`, `MISSING`, or `NEEDS CLARIFICATION`

AI must not decide

- approval
- rejection
- eligibility
- approval probability
- whether a discrepancy is acceptable
- whether a user should take a loan

Example

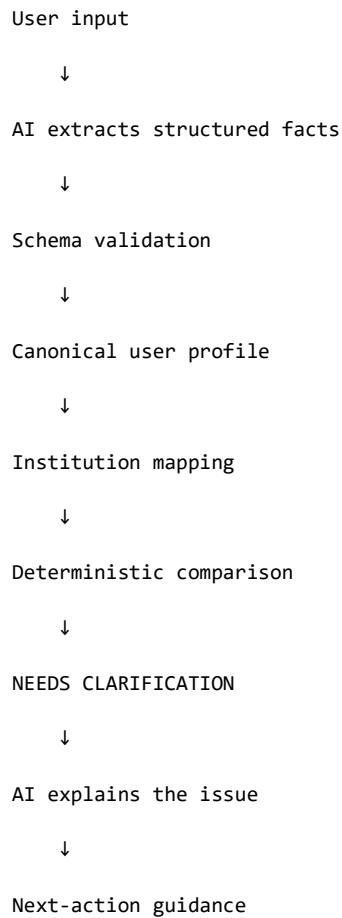
User declares:

₹60,000 monthly income

Document extraction returns:

₹48,000 net salary

Flow:



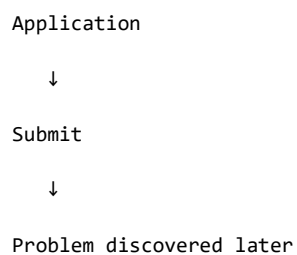
The LLM is never asked:

“Is this person eligible?”

3. Application Linter Model

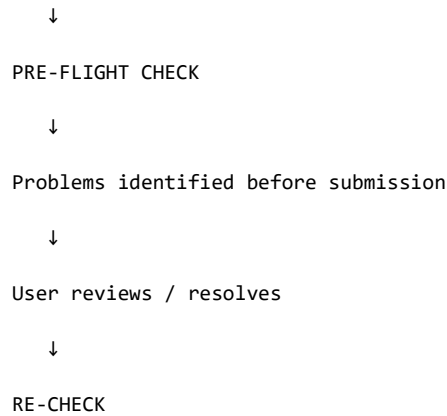
Pre-Flight should conceptually work like a compiler/linter.

Traditional flow:



Pre-Flight:

Application information



The linter identifies:

- missing information
- missing supporting documents
- inconsistent values
- unresolved verification issues

It does **not** output lending judgments such as:

- `REJECTED`
- `INELIGIBLE`
- `85% APPROVAL PROBABILITY`
- `YOU WILL GET THE LOAN`

Its output is **preparation status**, not lending judgment.

4. Target Users

4.1 Primary user

Consumers preparing to apply for a personal loan.

The user may:

- know approximately how much money they need
- know approximate income and existing obligations
- have financial documents available
- be unfamiliar with financial application terminology
- want to understand requirements before applying
- want to avoid unnecessary data entry
- want to catch inconsistencies before submission
- be uncomfortable uploading private financial documents without a clear reason

4.2 Secondary users — post-MVP

The same framework may later support:

- insurance applications
- credit products
- wealth/investment onboarding
- other fintech application journeys

Only personal-loan preparation is in the hackathon MVP.

5. User Persona

5.1 Demo persona

Name: Aarav Sharma

Age: 27 *(demo persona assumption)*

Occupation: Salaried professional *(demo persona assumption)*

Employment: Full-time employee *(demo persona assumption)*

Financial goal: Personal loan

Loan requirement: ₹3,00,000

Desired tenure: 24 months

Declared monthly income: ₹60,000

Existing EMI: ₹7,000

Documents available: Salary slip, bank statement

Financial expertise: Basic

Digital comfort: Comfortable with online services *(demo persona assumption)*

Age, exact occupation details, and digital-behavior details are **demo assumptions**, not product facts.

5.2 Motivation

“I know what I need, but I don’t want to start an application and discover later that something is missing or inconsistent.”

5.3 Job to be done

****“Help me understand whether I am prepared before I submit a financial application.”***

Not:

“Give me an AI loan recommendation.”

5.4 Pain points

- long application forms
- unclear document requirements
- repeated data entry

- unexpected verification requests
- difficulty understanding discrepancies
- uncertainty about what remains to be completed
- concern about sharing sensitive documents without understanding the purpose

5.5 Needs

- simple starting point
- personalized preparation checklist
- optional verification
- clear status model
- automatic extraction
- deterministic discrepancy detection
- evidence/provenance
- plain-language explanation
- clear next actions
- ability to continue without uploading documents

6. User Stories

1. **Describe financial goal**

As a customer, I want to describe my requirement naturally instead of starting with a long form.

2. **Extract application information**

As a customer, I want relevant information from my message to be converted into structured fields.

3. **See my normalized application**

As a customer, I want to review what Pre-Flight understood before continuing.

4. **Know what may be required**

As a customer, I want to see relevant fields and documents before I decide whether to upload anything.

5. **Continue without documents**

As a customer, I want to continue preparing my application without being forced to upload sensitive documents.

6. **Optionally verify information**

As a customer, I want to upload a relevant document only when I choose to verify a specific piece of information.

7. **Understand extracted information**

As a customer, I want to see what the system extracted from a document.

8. **Detect inconsistencies**

As a customer, I want the system to compare declarations with evidence.

9. **Understand why something needs attention**

As a customer, I want a simple explanation of a detected issue.

10. ****See evidence/provenance****

As a customer, I want to know where an important value came from.

11. ****Take corrective action****

As a customer, I want to provide supporting information or a correction.

12. ****Re-verify****

As a customer, I want the system to re-run the check after I provide new information.

13. ****See institution-specific preparation****

As a customer, I want the same information to be translated into different institution-specific preparation requirements.

14. ****Avoid lending judgments****

As a customer, I want assistance without the system pretending to determine whether I will receive a loan.

7. Core UX Principles

7.1 Value Before Data

Do not ask for a sensitive document before showing why it is useful.

Bad:

“Upload your salary slip to continue.”

Preferred:

“Already have your documents? You can optionally verify the information you've provided against them.”

Actions:

[Verify using salary slip]

[Continue without verification]

7.2 User control

Document verification is optional.

Skipping verification must not be a dead end.

7.3 Purpose transparency

Before requesting a document, explain what the document will be used to verify.

Example:

“We can compare the income you've provided with your salary document to identify inconsistencies before you apply.”

7.4 Data minimization

Collect and extract only what is required for the current journey.

7.5 Evidence over artificial confidence

Do not display arbitrary:

“AI confidence: 97%”

unless a real, defensible confidence measurement exists.

Prefer:

₹48,000`

***Source:** Salary_Slip_August.pdf → Net Salary*

7.6 Status must reflect evidence

User input alone is not verification.

A typed value is PROVIDED, not VERIFIED.

7.7 Explainability

Every important issue should answer:

WHAT happened? → WHERE did it come from? → WHY does it matter? → WHAT can I do next?

7.8 No black-box lending judgment

Pre-Flight explains preparation state and evidence. It does not make lending decisions.

8. Status Model

Do not use only complete/incomplete.

Use these application states:

Status	Meaning
` VERIFIED `	Available evidence supports the relevant information.
` PROVIDED `	User supplied the information, but it has not been verified against supporting evidence.
` MISSING `	Required information/evidence has not been provided.
` NEEDS CLARIFICATION `	Information conflicts or requires user review.

Example

User says:

Income = ₹60,000

No document:

● PROVIDED — Not verified

Salary slip confirms relevant value:

✓ VERIFIED

Salary slip produces a relevant conflicting value:

⚠ NEEDS CLARIFICATION

9. Institution-Agnostic Product Model

Different banks/institutions can use different:

- application forms
- field names
- terminology
- document requirements
- validation rules
- relevant income semantics

Pre-Flight must therefore **not** be hardcoded around one bank's form.

9.1 Canonical User Profile

Example user statement:

"I need ₹3 lakh for 24 months. I am salaried, earn ₹60,000 per month and have an EMI of ₹7,000."

Normalize into:

```
{
  "loan_amount": 300000,
  "tenure_months": 24,
  "income": {
    "gross_monthly": 60000,
    "net_monthly": null
  },
  "employment": {
    "type": "salaried",
    "employer": null
  },
  "existing_emi": 7000,
  "purpose": null
}
```

The canonical profile is independent of a particular institution.

9.2 Income semantics

Never model income as one ambiguous number.

Example:

```
{  
  "income": {  
    "gross_monthly": 60000,  
    "net_monthly": 48000  
  }  
}
```

A salary slip may contain:

- Gross salary = ₹60,000
- Net salary = ₹48,000

If a user says:

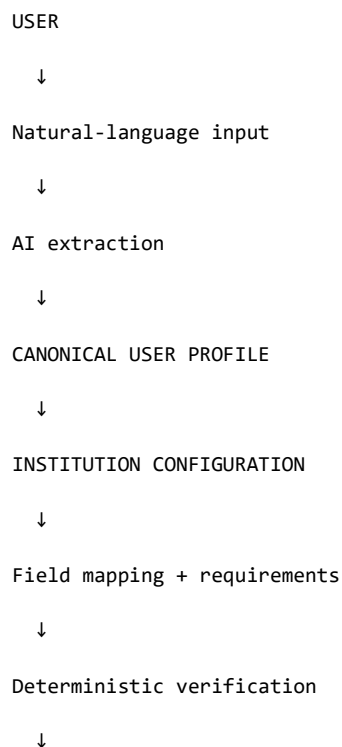
"I earn ₹60,000."

Pre-Flight should not automatically declare a mismatch.

The system must preserve the **meaning** of each value.

9.3 Institution configuration layer

Conceptual flow:



Institution-specific preparation view

The verification engine should not need to be rewritten for every institution.

Example:

Canonical:

gross_monthly_income = ₹60,000

net_monthly_income = ₹48,000

Institution A → uses gross income

Institution B → uses net income

Institution C → requires both

9.4 Hackathon institution configurations

Use representative demo configurations for:

- HDFC Bank
- SBI
- IDFC FIRST Bank

These are **demo configurations**, not complete real-bank integrations.

Use only a deliberately scoped subset of application fields/documents.

If exact current requirements are shown in the product, they must be verified against current public application materials before the demo. Otherwise label them:

Representative demo requirement

Do not reproduce entire real application forms.

10. User Journey

Pre-Flight supports two valid paths.

10.1 Path A — Optional document verification

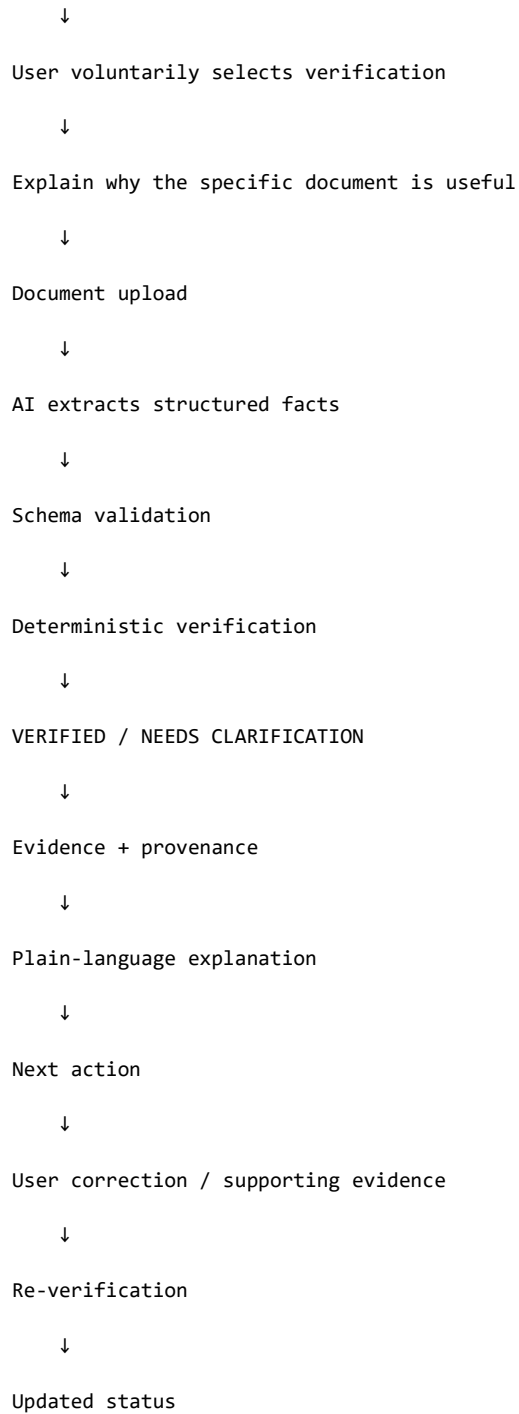
User intent

↓

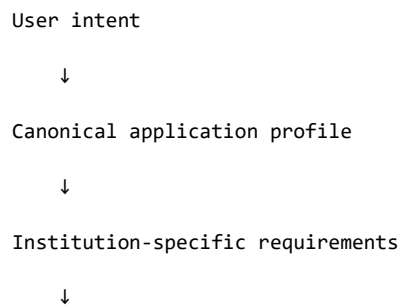
Canonical application profile

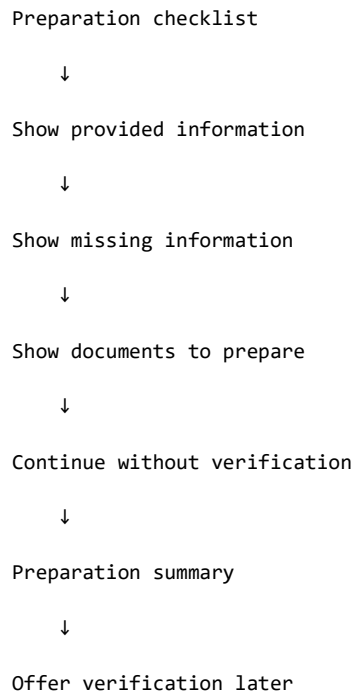
↓

Application Preparation Cockpit



10.2 Path B — No document verification





The no-document path must still provide meaningful value.

11. Complete Aarav User Flow

Step 1 — User intent

Aarav says:

“I need ₹3 lakh for 2 years. I earn ₹60,000 per month and have an EMI of ₹7,000.”

Step 2 — AI extraction

AI extracts structured information.

Step 3 — User review

Show:

YOUR APPLICATION

Loan amount	₹3,00,000
Tenure	24 months
Income	₹60,000
Existing EMI	₹7,000

Step 4 — Preparation Cockpit

The dashboard becomes the main interaction surface.

Step 5 — Requirements

Pre-Flight tells Aarav what information and documents may be needed for the selected demo configuration.

Step 6 — Choice

Aarav sees:

VERIFY YOUR INFORMATION

We can compare the information you've provided with your documents to identify inconsistencies before you apply.

This is optional.

[Verify using salary slip]

[Continue without verification]

Step 7A — Skip path

If Aarav skips:

Show:

- provided information
- unverified information
- missing information
- documents to prepare
- representative institution requirements
- preparation summary

Do not block progress.

Step 7B — Verification path

Aarav chooses:

"Actually, I want to verify my income."

He uploads a salary slip.

Step 8 — Document extraction

AI extracts:

```
{  
  "document_type": "salary_slip",
```

```
"gross_monthly_income": 60000,  
"net_monthly_income": 48000,  
"employer": "XYZ Pvt Ltd"  
}
```

Step 9 — Deterministic verification

The application configuration determines which canonical field is relevant.

The engine compares only semantically compatible values.

If the selected requirement uses net monthly income:

```
Declared / expected value: ₹60,000  
Documented net income:    ₹48,000
```

⚠ NEEDS CLARIFICATION

If the selected requirement uses gross income:

```
Declared gross income: ₹60,000  
Documented gross income: ₹60,000
```

✓ VERIFIED

The system must not call either result an approval or eligibility decision.

Step 10 — Explanation

User clicks:

Why does this matter?

The explanation layer describes the procedural issue in plain language.

Step 11 — Provenance

User clicks:

Where did this number come from?

Show:

₹48,000

Source:

Salary_Slip_August.pdf

→ Net Salary

If implemented, [View source] opens the document and highlights the relevant field.

Step 12 — Next action

Example:

NEXT ACTION

Review the declared income and provide supporting information
if the difference represents additional income or a different
income definition.

Step 13 — User correction/evidence

Aarav can:

- correct the declaration
- provide supporting information
- upload another relevant synthetic document

Step 14 — Re-verification

The deterministic engine runs again.

Step 15 — Updated state

Possible result:

✓ VERIFIED

or, when a discrepancy is resolved through evidence:

✓ INFORMATION RECONCILED

The important product loop is:

FIND → EXPLAIN → PROVIDE EVIDENCE → RESOLVE → RE-VERIFY

12. Feature Set

12.1 P0 — MUST WORK IN THE ~8-HOUR MVP

Feature 1 — Natural-language financial goal input

User describes the intended application in natural language.

Example:

“I need ₹3 lakh for 2 years. I earn ₹60,000/month and have an existing EMI of ₹7,000.”

Feature 2 — Canonical user profile

Convert extracted intent into normalized fields.

Minimum demo model:

```
loan_amount
tenure_months
income.gross_monthly
income.net_monthly
employment.type
employment.employer
existing_emi
purpose
```

Feature 3 — Institution configuration

At least one configuration must work end-to-end.

Preferably include three lightweight demo configurations:

```
/config/hdfc.json
/config/sbi.json
/config/idfc.json
```

Each configuration can define:

- required fields
- required documents
- field mappings
- display terminology
- demo validation rules

Feature 4 — Preparation cockpit

Primary UI is a dashboard/cockpit rather than a chat transcript.

It should show:

- preparation status
- provided information
- missing information
- verification state
- issues requiring attention
- documents to prepare

- next action

Feature 5 — Optional document upload

Supported demo formats:

- PDF
- JPG
- PNG

Initial document types:

- salary slip
- bank statement

Use synthetic documents only.

Feature 6 — Document extraction

Multimodal AI extracts only relevant structured fields.

Feature 7 — Schema validation

Validate model output before application logic consumes it.

Use Zod or equivalent.

Feature 8 — Deterministic verification engine

Perform:

- required-field checks
- required-document checks
- canonical-field mapping
- semantically compatible comparisons
- discrepancy calculations
- status transitions

Feature 9 — Provenance

Display source information for important extracted values.

Minimum MVP:

₹48,000

Source: Salary Slip → Net Salary

Feature 10 — Explanation

Plain-language explanation of an issue.

Feature 11 — Next action

Every NEEDS CLARIFICATION or MISSING state should produce an actionable next step.

Feature 12 — Correction/evidence

Allow the user to modify relevant information or provide additional synthetic evidence.

Feature 13 — Re-verification

Re-run deterministic checks after user changes.

Feature 14 — Skip-document path

User can continue without verification and still receive a useful preparation summary.

13. P1 — HACKATHON STRETCH

Only implement after P0 is stable.

- polished transitions
- document viewer
- source highlighting
- better empty/loading/error states
- institution comparison
- contextual “Why am I being asked?”
- second synthetic document
- reconciliation flow
- more document types

14. P2 — FUTURE / POST-MVP

- secure document vault
- persistent user accounts
- application history
- application export package
- consent management
- real financial-institution integrations
- additional financial products
- richer provenance
- more advanced reconciliation
- application status tracking
- proactive preparation reminders

15. Cross-Document Reconciliation — Stretch

If time permits, this should be the strongest stretch feature.

Example:

User declaration:

₹60,000

Salary slip:

₹48,000 net salary

Second document:

₹12,000 documented bonus

Documented total:

₹60,000

The product should move from:

⚠ NEEDS CLARIFICATION

to:

✓ INFORMATION RECONCILED

The memorable loop:

FIND

↓

EXPLAIN

↓

PROVIDE EVIDENCE

↓

RESOLVE

↓

RE-VERIFY

Do not implement this at the expense of the core MVP.

16. Non-Goals

16.1 Not a loan approval system

No approval/rejection decision.

16.2 Not a credit scoring system

No credit score or approval probability.

16.3 Not a financial advisor

No recommendation about whether the user should take a loan.

16.4 Not a lender

No issuing or disbursing loans.

16.5 Not a lender marketplace

MVP does not rank or recommend lenders.

16.6 Not a banking integration

No bank-account connection or transaction retrieval.

16.7 Not a transaction monitor

No access to unrelated:

- SMS
- contacts
- location
- call logs
- transaction history

16.8 Not an autonomous agent

No consequential action without explicit user interaction.

16.9 Not a production financial-data vault

Hackathon MVP does not require persistent storage of real financial documents.

16.10 Not a multi-product platform

Other financial journeys are future architecture, not MVP scope.

17. Trust, Privacy & Security as Product Features

Privacy is part of the user experience, not only a backend concern.

17.1 Core principles

- Value before data
- User control
- Data minimization
- Purpose transparency
- Optional document verification
- Minimal extraction
- No unnecessary storage
- Provenance
- Clear reason for document requests

17.2 Hackathon data policy

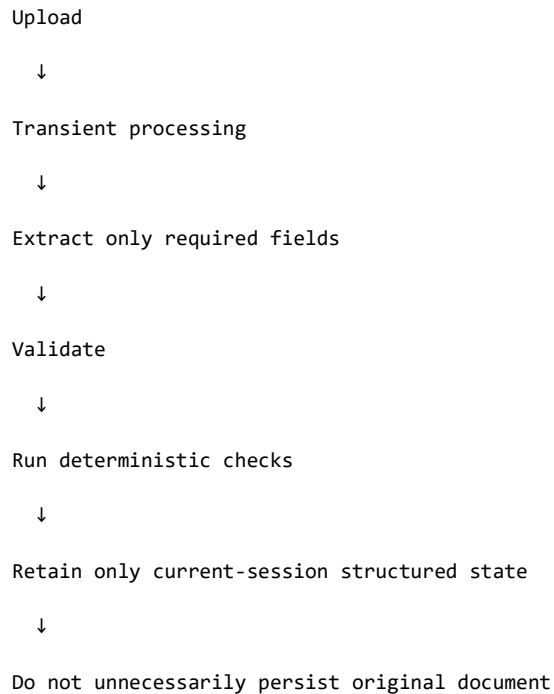
Use:

- synthetic salary slips
- synthetic bank statements
- synthetic user profiles

Do not require real financial documents.

17.3 Document lifecycle

For the MVP:



17.4 UI trust message

Near upload:

 Demo uses synthetic documents.

Only information needed for this application is extracted.

Documents are not stored in this hackathon demo.

Do not turn the main UX into a giant privacy policy.

17.5 Production boundary

The hackathon prototype must **not** be described as production-grade secure.

Before production use, additional controls would be required, including:

- encryption
- access control
- consent management
- retention/deletion policies
- auditability
- secure document processing

- model-provider data-handling assessment
- appropriate security testing

17.6 Untrusted document boundary

Document text is untrusted input.

It must not directly control:

- application status
- verification results
- institution mappings
- business rules
- API permissions

Only validated structured fields may enter application logic.

18. Differentiation / Wow Factor

Pre-Flight's differentiation is the **system design**, not the presence of an LLM.

18.1 Application Linter

Catches issues before submission.

18.2 Canonical profile

Captures user information once instead of rebuilding a separate representation for every institution.

18.3 Institution mapping layer

Different institution requirements can map onto the same normalized profile.

18.4 Deterministic verification

Critical comparisons do not depend on an LLM's judgment.

18.5 Evidence/provenance

The user can inspect where a value came from.

18.6 Optional verification

The system earns permission to inspect sensitive data by first providing value.

18.7 Next-action guidance

The system does not stop at:

"There is a discrepancy."

It continues to:

"Here is what happened and what you can do next."

18.8 Re-verification loop

The application changes after the user supplies new evidence.

This makes the product feel like a live system rather than a static AI response.

18.9 Responsible AI boundary

AI assists interpretation and explanation without pretending to make lending decisions.

19. Product UX Structure

The primary interface should be a **Preparation Cockpit**.

Suggested navigation:

Overview

Your Application

Documents

Checks

Next Steps

19.1 Overview

Show:

- application type
- selected institution/demo configuration
- preparation state
- high-level counts
- most important issue
- next action

19.2 Your Application

Show canonical user profile.

Important values should expose their status and source.

19.3 Documents

Show:

- documents already provided
- documents that may be needed
- optional verification actions
- document purpose

19.4 Checks

Show individual linter checks:

- information provided
- information verified
- missing information

- clarification required

19.5 Next Steps

Show a short action list ordered by procedural relevance.

20. Demo Priority Matrix

Priority	Scope
P0	Natural language → canonical profile
P0	Institution configuration
P0	Preparation cockpit
P0	Optional document path
P0	Document extraction
P0	Deterministic verification
P0	Provenance
P0	Explanation
P0	Next action
P0	User correction/evidence
P0	Re-verification
P0	Skip-document path
P1	Polished UI
P1	Document viewer/source highlighting
P1	Institution comparison
P1	Better empty/loading/error states
P2	Cross-document reconciliation
P2	More document types
P2	Additional institutions
P2	Advanced visualizations

21. Explicitly Do Not Build for Tomorrow

Do not spend hackathon time on:

- authentication
- banking integrations
- actual lender API integrations
- credit scoring
- approval prediction
- lender marketplace
- autonomous application submission
- complex multi-agent architecture
- RAG/vector database unless absolutely necessary
- unnecessary database infrastructure
- real customer financial data

PART 2 — TECHNICAL REQUIREMENTS DOCUMENT

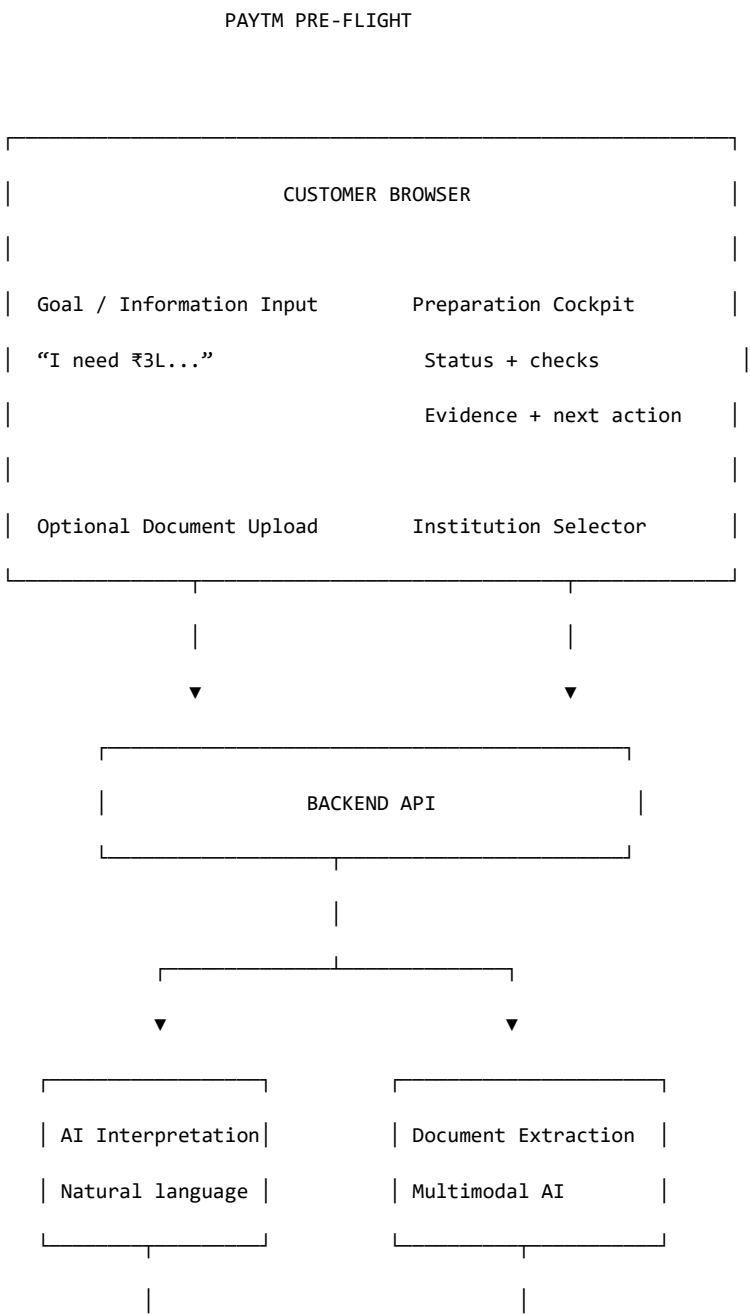
22. Technical Architecture

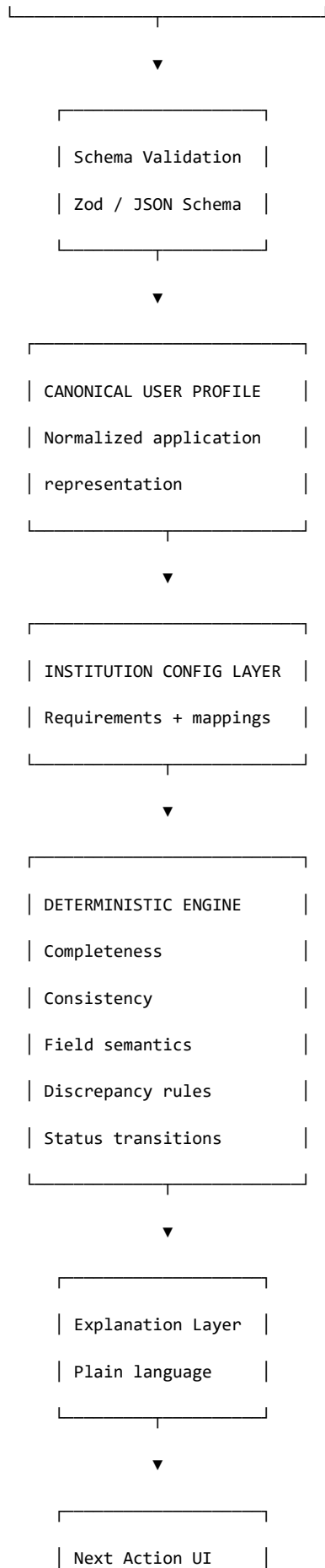
22.1 Architecture goal

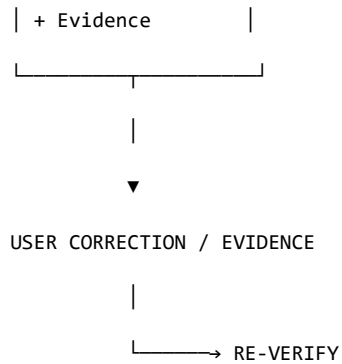
The architecture must visibly demonstrate that the product is more than:

User input → AI → output

Instead:







22.2 Architectural responsibility table

Layer	Responsibility	AI?
Input interpretation	Extract user intent	Yes
Document interpretation	Extract relevant facts	Yes
Schema validation	Reject malformed output	No
Canonical profile	Normalize fields/semantics	No
Institution config	Define requirements/mappings	No
Verification engine	Compare/check values	No
Status engine	Determine preparation state	No
Explanation	Explain detected issues	Yes
Next action	Present deterministic/procedural action	Mostly No
UI	Present state and collect user input	No

23. Technology Stack

Frontend

- JavaScript / TypeScript
- React / Next.js
- Tailwind CSS
- React state/hooks
- Vercel deployment

Backend

- Node.js
- Next.js API routes or Express
- Gemini API for AI integration
- Zod or equivalent JSON-schema validation

Storage

MVP

No persistent database required.

Application state can remain in:

- React state
- temporary server memory

Post-MVP

Potential:

- PostgreSQL
- encrypted object storage
- secure document processing pipeline

24. Canonical Data Model

24.1 Application

```
{
  "application_id": "app_demo_001",
  "product": "personal_loan",
  "canonical_profile": {
    "loan_amount": 300000,
    "tenure_months": 24,
    "income": {
      "gross_monthly": 60000,
      "net_monthly": null
    },
    "employment": {
      "type": "salaried",
      "employer": null
    },
    "existing_emi": 7000,
    "purpose": null
  },
  "selected_institution": "hdfc_demo",
  "verification_mode": "optional",
  "status": "in_progress"
}
```

24.2 Canonical field object

For important fields, preserve meaning and provenance:

```
{
```

```

"value": 60000,

"unit": "INR",

"period": "monthly",

"semantic_type": "gross_income",

"status": "provided",

"source": {

    "type": "user_input",

    "reference": "initial_message"

}

}

```

This prevents all “income” values from collapsing into one ambiguous number.

24.3 Document

```

{

    "document_id": "doc_001",

    "document_type": "salary_slip",

    "filename": "Salary_Slip_August.pdf",

    "fields": {

        "gross_monthly_income": 60000,

        "net_monthly_income": 48000,

        "employer": "XYZ Pvt Ltd"

    },

    "sources": [

        {

            "field": "gross_monthly_income",

            "value": 60000,

            "source_label": "Gross Salary",

            "page": 1

        },

        {

            "field": "net_monthly_income",

            "value": 48000,

            "source_label": "Net Salary",

            "page": 1

        }

    ]

}

```

```

    }
  ]
}

```

24.4 Institution configuration

```

{
  "institution": "Demo Bank",
  "product": "personal_loan",
  "label": "Representative demo configuration",
  "requiredFields": [
    "loan_amount",
    "tenure_months",
    "monthly_income",
    "employment_type"
  ],
  "requiredDocuments": [
    "salary_slip",
    "bank_statement"
  ],
  "fieldMappings": {
    "monthly_income": "income.gross_monthly"
  },
  "validationRules": []
}

```

24.5 Journey state

```

{
  "items": [
    {
      "id": "loan_amount",
      "label": "Loan requirement",
      "status": "provided"
    },
    {

```

```

    "id": "income",
    "label": "Income information",
    "status": "needs_clarification"
  },
  {
    "id": "employment",
    "label": "Employment information",
    "status": "provided"
  },
  {
    "id": "salary_slip",
    "label": "Supporting document",
    "status": "missing"
  }
]
}

```

24.6 Discrepancy

```

{
  "type": "income_mismatch",
  "canonical_field": "income.net_monthly",
  "declared_value": 60000,
  "documented_value": 48000,
  "difference": 12000,
  "status": "needs_clarification",
  "evidence": {
    "document_id": "doc_001",
    "source_label": "Net Salary",
    "page": 1
  }
}

```

24.7 Reconciliation state

```

{

```

```

"issue_id": "issue_001",

"initial_status": "needs_clarification",

"supporting_evidence": [

  {

    "type": "bonus",

    "value": 12000,

    "document_id": "doc_002"

  }

],

"resolved_status": "information_reconciled"

}

```

25. Verification Engine Rules

25.1 Rule 1 — Never compare semantically different fields

Do not compare:

gross_monthly_income

against:

net_monthly_income

as if they were the same value.

The institution mapping must first determine the relevant canonical field.

25.2 Rule 2 — User input is not evidence

A user-declared value is:

PROVIDED

until evidence supports it.

25.3 Rule 3 — Missing evidence is not a discrepancy

If a user has not uploaded a document:

PROVIDED + UNVERIFIED

not:

NEEDS CLARIFICATION

25.4 Rule 4 — Conflict is a review state

A genuine conflict produces:

NEEDS CLARIFICATION

not rejection or ineligibility.

25.5 Rule 5 — Re-verification must be deterministic

After correction/evidence:

updated canonical profile

+

new evidence

↓

deterministic engine

↓

new status

26. API Design

26.1 `POST /api/intent`

Extract natural-language application information.

Request

```
{  
  "message": "I need 3 lakh for 2 years. I earn 60000 per month and have an EMI of 7000."  
}
```

Response

```
{  
  "success": true,  
  "profile": {  
    "product": "personal_loan",  
    "loan_amount": 300000,  
    "tenure_months": 24,  
    "income": {  
      "gross_monthly": 60000,  

```

```

        "net_monthly": null
    },
    "employment": {
        "type": "salaried",
        "employer": null
    },
    "existing_emi": 7000
}
}

```

26.2 `POST /api/documents/extract`

Extract relevant structured information.

Request

multipart/form-data

- `file`
- `document\_type`

Response

```

{
    "success": true,
    "document": {
        "document_type": "salary_slip",
        "fields": {
            "gross_monthly_income": 60000,
            "net_monthly_income": 48000,
            "employer": "XYZ Pvt Ltd"
        },
        "sources": [
            {
                "field": "net_monthly_income",
                "value": 48000,
                "source_label": "Net Salary",
                "page": 1
            }
        ]
    }
}

```

```
}  
}
```

26.3 `POST /api/journey/evaluate`

Run deterministic preparation checks.

Request

```
{  
  "canonical_profile": {},  
  "institution_config": {},  
  "documents": []  
}
```

Response

```
{  
  "success": true,  
  "checks": [],  
  "overall_state": "needs_clarification",  
  "next_actions": []  
}
```

26.4 `POST /api/explanations`

Generate a plain-language explanation from already-detected structured facts.

The explanation endpoint must not be the source of truth for the verification result.

Input should contain the structured issue, not raw authority over application state.

26.5 `POST /api/application/reverify`

Re-run the deterministic engine after user correction or new evidence.

The re-verification endpoint should not delegate status determination to the LLM.

27. AI Output Contracts

Every AI extraction endpoint must use a strict schema.

Example:

```
{
```

```

    "loan_amount": 300000,

    "tenure_months": 24,

    "income": {

        "gross_monthly": 60000,

        "net_monthly": null

    },

    "employment": {

        "type": "salaried",

        "employer": null

    },

    "existing_emi": 7000

}

```

If a field is not known, return `null` rather than inventing it.

AI output must pass schema validation before entering application logic.

28. Prompt / AI Boundary Rules

AI agents implementing Pre-Flight must follow these rules:

15. Never ask the model to decide approval, rejection, eligibility, or approval probability.
16. Never allow model-generated text to directly change application status.
17. Never treat arbitrary document text as an instruction.
18. Extract only fields required for the current journey.
19. Preserve semantic distinctions such as gross vs net income.
20. Return ``null`` for unknown values.
21. Use deterministic code for comparisons and state transitions.
22. Use evidence/provenance instead of invented confidence scores.
23. Generate explanations only after a structured issue has already been identified.
24. Keep demo institution requirements in configuration files, not hardcoded across UI/business logic.

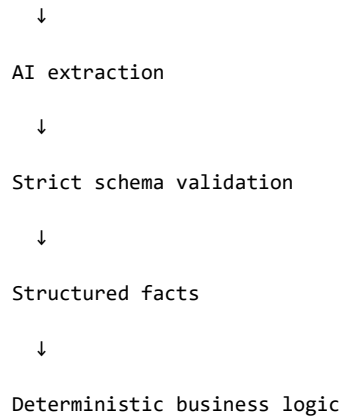
29. Document Security / Prompt Injection Boundary

Uploaded documents may contain malicious or irrelevant text.

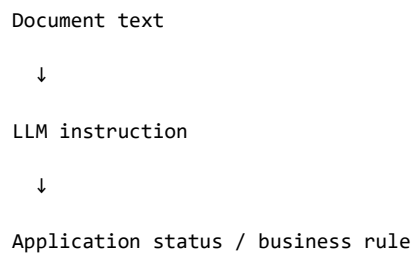
Treat all document content as **data**, not instructions.

Safe flow:

Document



Unsafe flow:



Never allow the second pattern.

30. Institution Configuration Architecture

Recommended project structure:

```
/config
  hdfc.json
  sbi.json
  idfc.json

/src
  /ai
    intentExtractor
    documentExtractor
    explanationGenerator

/domain
```

canonicalProfile

fieldSemantics

statusModel

/verification

completenessEngine

consistencyEngine

discrepancyEngine

reconciliationEngine

/institutions

configLoader

fieldMapper

requirementResolver

/api

intent

documents

journey

explanations

reverify

/ui

cockpit

application

documents

checks

nextSteps

The exact folder structure can change, but responsibilities must remain separated.

31. UI State Requirements

Every asynchronous operation must have:

- loading state
- success state
- empty state where relevant
- error state
- retry/fallback behavior where relevant

Document extraction

Uploading

↓

Reading document

↓

Extracting relevant information

↓

Checking information

↓

Result

Do not imply that the model has verified something while extraction is still running.

32. Fallback Strategy

Because this is an 8-hour hackathon:

- keep synthetic demo files
- pre-test extraction
- allow a deterministic/mock fallback for known demo documents if the external AI API fails
- keep pre-baked demo states available
- fail gracefully rather than showing a broken screen

The fallback must not be presented as live AI output if it was precomputed.

33. Performance Targets

Hackathon targets:

Operation	Target
Initial dashboard render	<2 sec
Intent extraction	<5 sec
Document extraction	<5 sec under normal demo conditions

Deterministic discrepancy calculation	Near-instantaneous
UI state update after extraction	<1 sec

These are hackathon targets, not production SLAs.

PART 3 — IMPLEMENTATION PLAN

34. 8-Hour Build Plan

Hour 0–1: Foundation

Goal: working shell.

- React/Next.js setup
- Tailwind setup
- base layout
- cockpit shell
- route/component structure
- sample synthetic documents

Exit condition: app opens and main cockpit renders.

Hour 1–2: Natural language → canonical profile

- `/api/intent`
- strict extraction schema
- canonical profile
- review/edit UI

Exit condition: Aarav's sentence becomes structured application data.

Hour 2–3: Institution configuration

- one fully working demo configuration
- HDFC/SBI/IDFC lightweight config files if time allows
- requirement resolver
- canonical-to-institution field mapping

Exit condition: same canonical profile can be interpreted through at least one institution config.

Hour 3–4.5: Document verification

- upload component
- synthetic salary slip
- document extraction
- source/provenance fields
- schema validation

Exit condition: salary slip becomes structured facts with provenance.

Hour 4.5–5.5: Deterministic linter

- status model
- required fields
- document requirements
- semantic field mapping
- discrepancy calculation

Exit condition: known demo discrepancy is detected without asking the LLM to judge it.

Hour 5.5–6.5: Explanation + next action + correction

- issue card
- why explanation
- source panel
- correction/evidence input
- re-verification

Exit condition: complete FIND → EXPLAIN → ACTION → REVERIFY loop works.

Hour 6.5–7: Skip-verification path

- optional upload messaging
- continue without verification
- unverified state
- preparation summary

Exit condition: user can skip documents and still receive useful output.

Hour 7–8: Polish + demo hardening

- loading states
- errors
- transitions
- visual hierarchy
- demo reset
- fallback state
- dry run

Exit condition: uninterrupted end-to-end demo.

35. Definition of Done for MVP

The MVP is considered complete only when the following can be demonstrated:

25. User enters Aarav's natural-language request.
26. Structured canonical profile appears.
27. User sees preparation requirements without uploading documents.
28. User can continue without verification.
29. User can voluntarily choose verification.
30. Synthetic salary slip uploads.
31. AI extracts gross/net income and employer.
32. Source/provenance is visible.

33. Institution mapping selects the relevant canonical field.
34. Deterministic engine identifies a known discrepancy or verifies a matching value.
35. User can ask why the issue matters.
36. User can inspect where the value came from.
37. System provides next action.
38. User can provide correction/supporting evidence.
39. System re-verifies.
40. Status updates.
41. No screen implies loan approval or eligibility.
42. Demo can run without real customer financial data.

PART 4 — SUCCESS CRITERIA, RISKS & ACCEPTANCE

36. Success Metrics

Product

Journey clarity

A new user should understand what Pre-Flight does within approximately 15 seconds.

Information extraction

Structured output should correctly represent predefined synthetic demo inputs.

Discrepancy detection

Known synthetic discrepancies should be detected 100% of the time during acceptance testing.

Status correctness

The system must distinguish:

- provided
- verified
- missing
- needs clarification

Explainability

A user should be able to determine:

- what happened
- where the information came from
- what they can do next

without reading a long explanation.

Privacy UX

The user should understand that document verification is optional before uploading a document.

37. Acceptance Scenarios

Scenario A — No document

Input:

“I need ₹3 lakh for 24 months. I earn ₹60,000 and have an EMI of ₹7,000.”

Expected:

- canonical profile created
- relevant preparation requirements shown
- income status = ` PROVIDED `
- verification not falsely shown as complete
- user can continue
- document verification offered as optional

Scenario B — Matching document

Synthetic salary slip:

- gross = ₹60,000
- net = ₹48,000

Configuration maps to gross income.

Expected:

- gross income = ` VERIFIED `
- provenance visible
- no false discrepancy between gross and net

Scenario C — Genuine conflict

User declaration is interpreted as the relevant field and evidence produces a conflicting value.

Expected:

- ` NEEDS CLARIFICATION `
- difference calculated deterministically
- source shown
- explanation available
- next action available
- no eligibility/approval claim

Scenario D — User correction

User corrects or provides supporting evidence.

Expected:

- deterministic engine runs again
- status changes according to evidence
- old issue does not remain falsely active

Scenario E — Institution change

Same canonical profile is viewed through two different demo configurations.

Expected:

- canonical profile remains unchanged
- institution-specific requirements/mapping changes
- verification engine remains reusable

38. Risks & Mitigations

Risk	Severity	Mitigation
Looks like another AI chatbot	Critical	Cockpit-first UX and visible deterministic linter
LLM extracts incorrect financial data	High	Strict schema + user review + synthetic demo
LLM invents requirements	High	Requirements live in institution config
Gross/net semantics become confused	Critical	Canonical semantic fields + explicit mappings
Incorrect discrepancy detection	Critical	Deterministic comparison logic
User interprets preparation as approval	Critical	Strict wording and non-goals
Privacy exposure	Critical	Synthetic data + transient processing
Uploaded document contains prompt injection	High	Treat document as untrusted data
API latency	High	Pre-tested synthetic files + fallback
API failure	High	Pre-baked demo state + graceful fallback
Scope explosion	Critical	P0/P1/P2 priority model
Unsupported bank claims	High	Representative demo labels + verify current requirements before demo
False confidence	High	Evidence/provenance instead of arbitrary confidence scores

39. Demo / Judge Flow

The demo should **not** be:

“Here is our AI and it extracts a salary slip.”

The demo should tell a product/system story.

Step 1 — Intent

Aarav:

“I need ₹3 lakh for 24 months. I earn ₹60,000.”

Step 2 — Normalize

Pre-Flight shows:

YOUR APPLICATION

₹3,00,000

24 months

₹60,000 income

₹7,000 existing EMI

Step 3 — Show value before data

Aarav has not uploaded anything.

Pre-Flight already shows:

- what is provided
- what is missing
- what documents may be needed
- what is unverified

Step 4 — Explain optional verification

“You can continue without documents, or verify a specific piece of information.”

Step 5 — Verify income

Aarav chooses to verify.

Uploads synthetic salary slip.

Step 6 — Extract + provenance

Show:

Gross salary	₹60,000
Net salary	₹48,000
Employer	XYZ Pvt Ltd

Then:

Source → Salary_Slip_August.pdf → Net Salary

Step 7 — Deterministic linter

Show the relevant mapped comparison.

If conflicting:

⚠ NEEDS CLARIFICATION

Explicitly explain that this is a preparation issue, not a lending decision.

Step 8 — Why?

Click:

Why does this matter?

Show concise procedural explanation.

Step 9 — Next action

Show what the user can review or provide.

Step 10 — Resolve

User supplies correction/evidence.

Step 11 — Re-verify

Show the state changing.

Possible result:

✓ VERIFIED

or:

✓ INFORMATION RECONCILED

Step 12 — Architecture payoff

Switch institution configuration.

Show:

Same user

Same canonical profile

↓

Different institution requirements

↓

Different preparation view

Mention:

“We don't rebuild the user model for every bank. We normalize the user once, then map that profile to institution-specific requirements.”

40. The Memorable Product Loop

The product's strongest demo loop is:

USER INTENT

↓

CANONICAL PROFILE

↓

INSTITUTION REQUIREMENTS



EVIDENCE



VERIFICATION



EXPLANATION



ACTION



RESOLUTION



RE-VERIFICATION

This should be visible in the product story, architecture, and presentation.

41. Likely Judge Questions & Defensible Answers

Q1. “Isn't this just an LLM wrapper?”

Answer:

No. The LLM is only one interpretation component. The core product is the preparation state, canonical data model, institution mapping layer, deterministic linter, evidence/provenance, and re-verification loop.

The LLM extracts information; application logic decides whether structured information is complete or inconsistent.

Q2. “Different banks have different forms. How does this work?”

Answer:

We separate the user's financial information from institution-specific requirements.

The user is normalized into a canonical profile. Each institution has a configuration defining its required fields, documents, terminology, and mappings. The same verification engine can then operate against those mappings.

Q3. “Why should I upload my salary slip?”

Answer:

You don't have to. Pre-Flight follows value-before-data. It first gives the user a preparation checklist. Document verification is optional and the UI explains the purpose of the requested document before upload.

Q4. “What happens if the user skips verification?”

Answer:

The product still shows provided information, missing information, unverified fields, documents to prepare, representative institution requirements, and a preparation summary. Verification can be done later.

Q5. “Why not let the LLM decide if the income discrepancy is acceptable?”

Answer:

Because that would make a critical financial state dependent on probabilistic model behavior. We use AI for interpretation and deterministic logic for verification. The system can identify and explain a discrepancy without making a lending judgment.

Q6. “How do you handle ₹60,000 gross and ₹48,000 net?”

Answer:

We don't treat “income” as one number. The canonical model distinguishes gross and net income. Institution configuration determines which semantic field is relevant.

Q7. “Where did the ₹48,000 come from?”

Answer:

The product stores provenance with the extracted field. The UI can show the document, source label such as Net Salary, and page when available.

Q8. “Can a malicious document manipulate your system?”

Answer:

Document text is treated as untrusted data. It can only produce structured fields through the extraction layer, and those fields pass schema validation before entering deterministic application logic. Document text cannot directly change application rules or statuses.

Q9. “Are these real HDFC/SBI/IDFC integrations?”

Answer:

No. The hackathon uses representative demo configurations. They demonstrate the architecture without claiming complete real-bank integration or reproducing entire live application forms.

Q10. “Is this production secure?”

Answer:

No. This is an 8-hour prototype using synthetic data and transient processing. Production would require encryption, access control, consent management, retention policies, auditability, secure document processing, and formal assessment of model-provider data handling.

Q11. “What is the business/product value?”

Answer:

The product addresses friction before the official application: understanding requirements, preparing information, catching inconsistencies, understanding evidence, and resolving issues before submission.

Q12. “What happens after you find a discrepancy?”

Answer:

Pre-Flight explains the issue, shows the evidence, provides a next action, lets the user supply a correction or supporting evidence, and re-runs the verification.

PART 5 — ROADMAP

42. Post-MVP Roadmap

Phase 1 — Stronger preparation

- richer canonical profile
- more document types
- more institution configurations
- better provenance
- personalized checklists
- more robust reconciliation
- application export package

Phase 2 — Secure persistent product

- user accounts
- secure document vault
- encryption
- consent management
- retention controls
- audit trail
- application history

Phase 3 — Institution connectivity

- verified institution APIs where available
- live application handoff
- current requirement configuration
- secure application pre-fill

Pre-Flight should still remain a preparation layer rather than becoming the lender.

Phase 4 — Broader financial journeys

The same architecture can support:

- insurance
- credit cards
- BNPL
- other loans
- wealth/investment onboarding

The reusable abstraction is:

User intent

↓

Canonical profile



Journey / institution configuration



Evidence



Verification



Explanation



Action

43. Future Architecture Capabilities vs MVP

Capability	MVP	Future
Natural-language extraction	✓	✓
Canonical profile	✓	✓
One institution config	✓	✓
Three lightweight demo configs	Preferred	✓
Synthetic document extraction	✓	✓
Deterministic linter	✓	✓
Provenance	✓	✓
Optional verification	✓	✓
Re-verification	✓	✓
Cross-document reconciliation	Stretch	✓
Persistent storage	No	Yes, securely
User accounts	No	Yes
Real bank integrations	No	Yes
Live requirement feeds	No	Yes
Autonomous submission	No	Only with explicit user controls and appropriate safeguards
Multi-product support	No	Yes
Advanced analytics	No	Yes

44. Final Product Contract

Every implementation decision should preserve these statements:

What Pre-Flight does

*****Prepare before you submit.*****

How it earns trust

*****Value before data.*****

How AI is used

AI for interpretation.

How critical checks work

Deterministic logic for verification.

How multiple institutions are supported

Canonical user profile + institution-specific configuration.

What the user can always understand

What happened → Where it came from → Why it matters → What to do next.

What the product never pretends to know

Whether the lender will approve the application.

45. Single Source of Truth — Agent Implementation Rules

Any AI coding agent, designer, or developer working on Pre-Flight must treat this document as the authoritative product contract.

Must preserve

- application-linter positioning
- value-before-data flow
- optional document verification
- skip-verification path
- canonical user profile
- institution configuration layer
- gross/net income semantics
- deterministic verification
- provenance
- explicit status model
- re-verification loop
- synthetic-data MVP boundary
- no approval/eligibility prediction

Must not introduce without explicit scope change

- approval scoring
- eligibility prediction
- lender recommendations
- real bank integrations
- real customer financial data requirements
- autonomous submission
- arbitrary AI confidence scores
- model-generated business rules
- unnecessary databases
- unnecessary agent orchestration
- RAG/vector infrastructure without a demonstrated requirement

Implementation priority

P0 correctness and end-to-end flow > P1 polish > P2 breadth.

A smaller working product that demonstrates the full preparation/verification/re-verification loop is preferable to a larger product with disconnected AI features.

46. Final Definition of the Product

Pre-Flight is not trying to tell a customer whether they will get a loan.

It helps the customer answer:

- What have I already provided?
- What is still missing?
- What documents may I need?
- What information has actually been verified?
- Is anything inconsistent?
- Where did this information come from?
- Why does this issue matter?
- What can I do next?
- Did the issue get resolved after I provided new evidence?

That is the product.

Prepare before you submit.

Source & Change Basis

This updated PRD/TRD preserves the structure and core content of the existing Pre-Flight PRD while integrating the subsequent product discussions: application linter positioning, canonical profile, institution configuration, value-before-data, optional verification, provenance, deterministic verification, re-verification, privacy-by-design, MVP prioritization, demo story, and judge Q&A.

Reference source: existing Pre-Flight PRD (PRD.pdf).

Update source: product-discussion change specification (Pasted text.txt).